

Propagacao de Fake News em Sistemas Paralelos e Distribuidos

Sequencial · Paralela (Threads) · Distribuida (Sockets)

Melhorias: Influenciadores Digitais · Resistencia a Propagacao

Disciplina: Sistemas Distribuidos

Integrantes: _____ (preencher)

1. Descricao do Problema

Simular como uma **fake news** se propaga numa populacao representada por uma matriz bidimensional $N \times M$, inspirada em **automatos celulares** e na dinamica de difusao de informacao em redes.

Estados de cada individuo (celula):

Estado	Codigo	Significado
Ignorante	0	Ainda nao acredita / nao recebeu
Espalhador	1	Acredita e compartilha ativamente
Inativo	2	Recebeu, mas parou de compartilhar

A propagacao ocorre **localmente** pela vizinhanca de Moore (ate 8 vizinhos), em geracoes discretas.

2. Modelo Computacional

Regras de transicao (aplicadas a TODAS as celulas a cada geracao):

- **Ignorante** → **Espalhador**: se o nº de vizinhos espalhadores $\geq$ limiar de convencimento
- **Espalhador** → **Inativo**: sempre na geracao seguinte (duracao de 1 geracao)
- **Inativo** → **Inativo**: permanente (nao volta a propagar)

Vizinhanca de Moore (X = celula central):

```
A B C → os 8 vizinhos de X
D X E
F G H
```

Propriedade-chave: a proxima geracao e calculada em uma **nova matriz**, lendo sempre o estado da geracao atual (atualizacao sincrona). Isso torna o resultado **determinista** e independente da ordem de processamento — o que permite paralelizar sem alterar o comportamento logico.

3. Solucao Sequencial (baseline)

Percorre a matriz inteira em um unico fluxo, celula a celula, gerando a nova matriz. Serve de referencia para speedup/eficiencia.

Nucleo do laco (arquivo FakeNews_sequencial.py):

```
for i in range(linhas):
    for j in range(colunas):
        nova[i][j] = proxima_celula(grade, i, j, limiar)
```

Complexidade por geracao: $O(\text{linhas} \times \text{colunas} \times 8)$. O custo cresce linearmente com o nº de celulas — motivacao para paralelizar.

4. Versao Paralela (Threads)

Divisao explicita: a matriz e fatiada em **faixas horizontais** contiguas, uma por thread (FakeNews_paralelo.py).

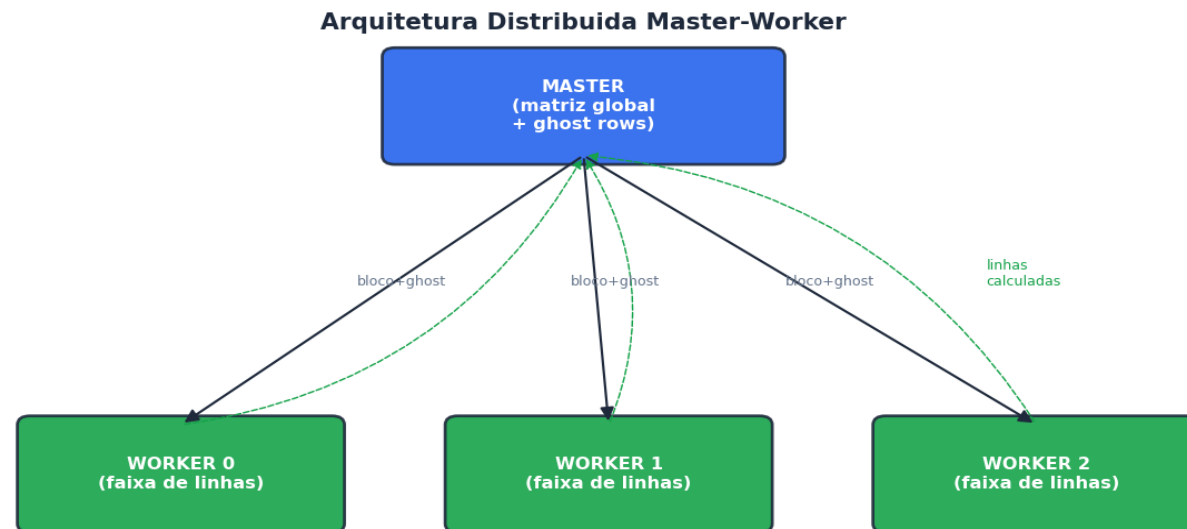
- Cada thread calcula apenas SUAS linhas, lendo a grade atual (somente leitura) e escrevendo na sua regio de 'nova_grade'.
- Regioes de escrita **disjuntas** $\Rightarrow$ sem condicao de corrida (race condition), sem lock.
- Sincronizacao por **threading.Barrier**: ao fim de cada geracao a barreira dispara a troca de grades e a verificacao de parada $\Rightarrow$ consistencia entre geracoes.
- **Threads persistentes**: criadas uma vez, processam todas as geracoes (evita overhead).

Limitacao fundamental (analizada na secao 8): o **GIL** do CPython serializa bytecode Python, limitando o ganho de threads para trabalho CPU-bound puro.

5. Versao Distribuida (Sockets)

Arquitetura **Master-Worker** sobre TCP (FakeNews_servidor.py + FakeNews_worker.py). Cada worker e um **processo independente** (sem GIL compartilhado).

- O master divide as linhas em faixas (uma por worker) e, a cada geracao, envia a cada um o seu bloco + as **ghost rows** (linhas vizinhas emprestadas).
- As ghost rows **sincronizam as fronteiras**: sem elas, a 1ª e a ultima linha de cada faixa ficariam inconsistentes em relacao a versao sequencial.
- O worker calcula apenas suas linhas reais e devolve; o master remonta a matriz global $\Rightarrow$ consistencia de geracao garantida.
- Protocolo: JSON com **prefixo de tamanho (4 bytes)** para delimitar mensagens no fluxo TCP.



Comunicacao TCP via sockets — JSON com prefixo de tamanho (4 bytes)

6. Divisao da Matriz / Processamento

Ambas as versoes usam **particionamento por faixas de linhas** (decomposicao de dominio 1D):

Aspecto	Paralela (Threads)	Distribuida (Sockets)
Unidade	Thread	Processo (worker)
Memoria	Compartilhada	Isolada (copia por mensagem)
Fronteiras	Leitura direta da grade	Ghost rows via rede
Sincronizacao	threading.Barrier	Master junta as faixas
Comunicacao	Nenhuma (memoria)	TCP (JSON + prefixo)
Limite de CPU	GIL (CPython)	Real (processos)

A equivalencia foi **verificada** (testar_corretude.py): grade final e historico identicos a versao sequencial em 1, 2, 4 e 8 threads e 1, 2, 3 workers, inclusive em grades com dimensoes nao multiplas do nº de unidades.

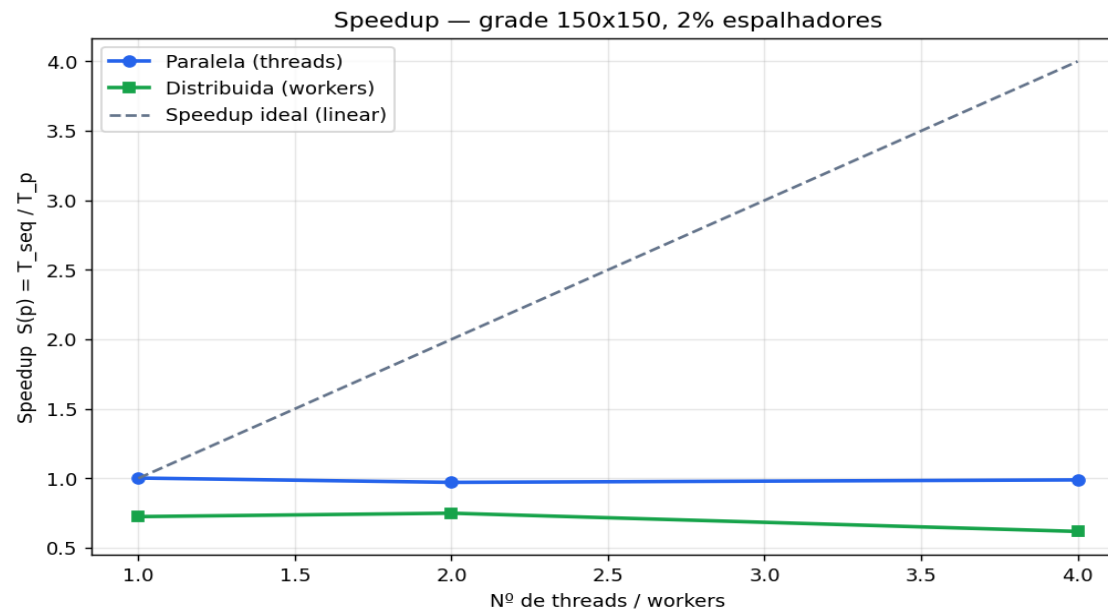
7. Resultados Experimentais

Grade 150x150 — 2% espalhadores

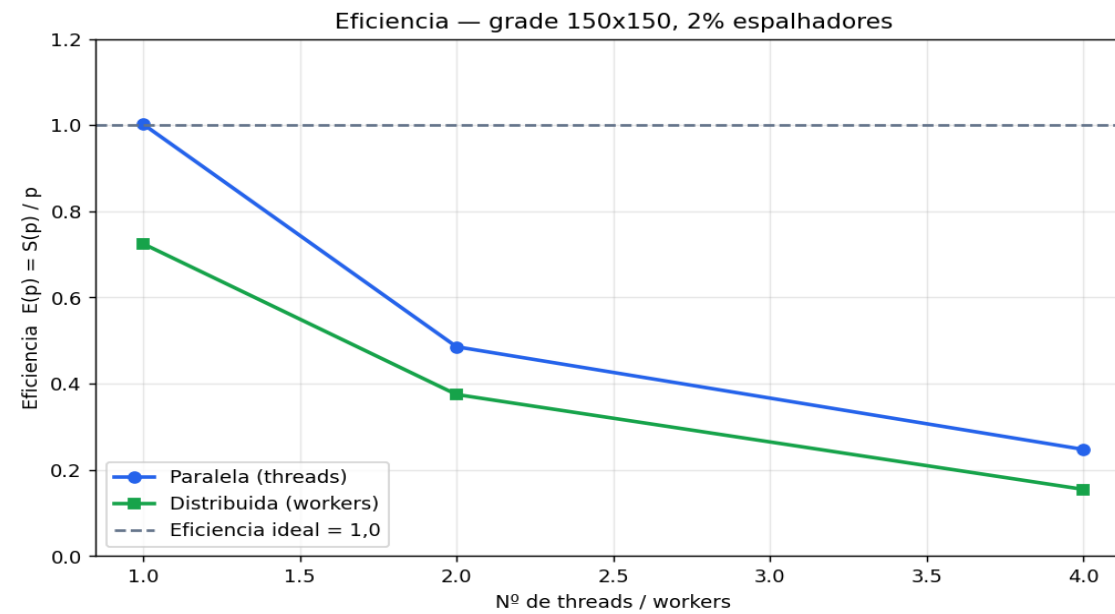
Versao	Threads/Workers	Tempo	Speedup	Eficiencia
Distribuida	1	82.0 ms	0.72	0.72
Distribuida	2	79.3 ms	0.75	0.37
Distribuida	4	96.2 ms	0.62	0.15
Paralela	1	59.3 ms	1.00	1.00
Paralela	2	61.2 ms	0.97	0.49
Paralela	4	60.1 ms	0.99	0.25
Sequencial	1	59.5 ms	1.00	1.00

Atencao: os numeros acima foram gerados em ambiente de build de **1 nucleo**. Regenere na maquina multi-core do grupo (benchmark.py → gerar_graficos.py → gerar_apresentacao.py) para valores representativos.

7.1 Speedup e Eficiencia

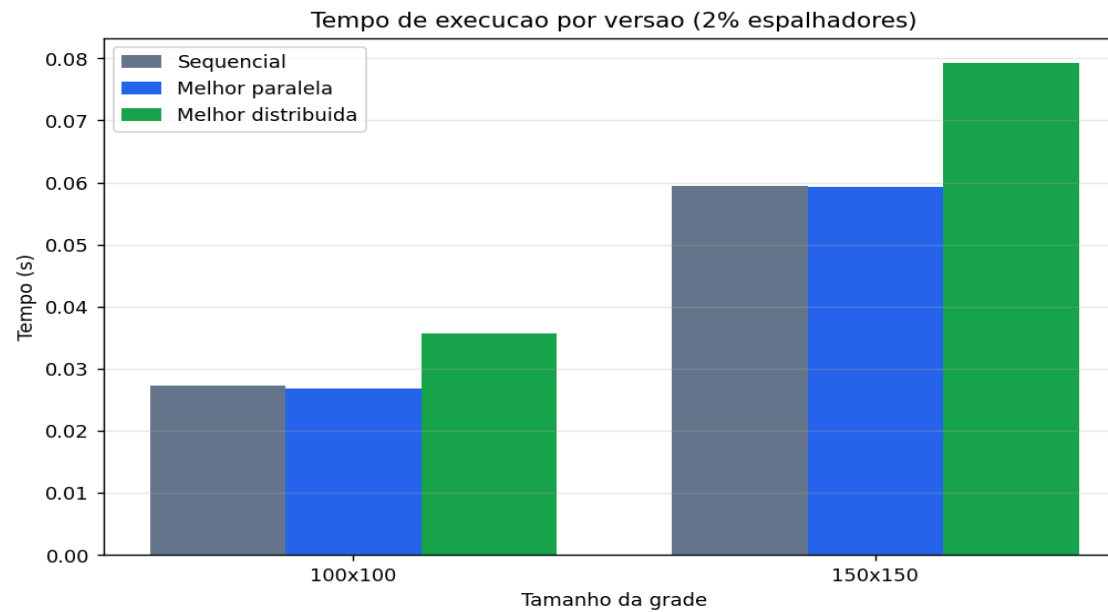


Ambiente de build: 1 nucleo. Regenere na maquina multi-core do grupo para resultados representativos.

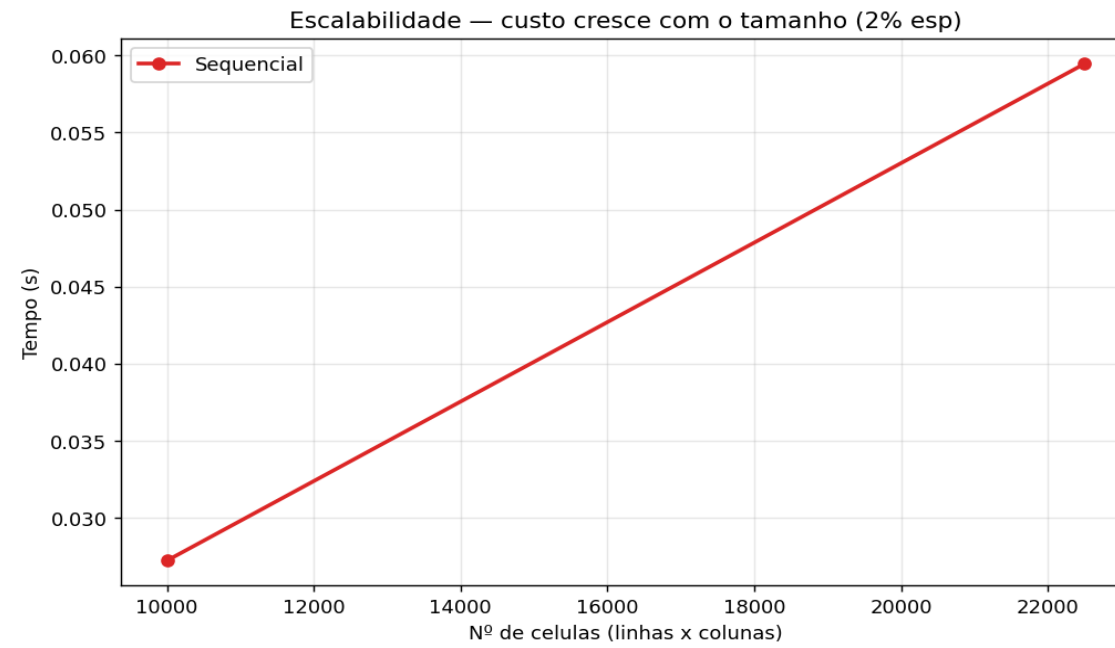


Ambiente de build: 1 nucleo. Regenere na maquina multi-core do grupo para resultados representativos.

7.2 Tempo por versao e escalabilidade



Ambiente de build: 1 nucleo. Regenere na maquina multi-core do grupo para resultados representativos.



8. Analise de Gargalos e Dificuldades

- **GIL (paralela):** em CPython, threads não executam bytecode Python em paralelo. Para trabalho CPU-bound, o speedup com threads tende a ~1. A divisão e os resultados estão corretos, mas o ganho de CPU é limitado pela plataforma — não pela lógica.
- **Custo de comunicação (distribuída):** serializar JSON e trafegar a matriz por geração domina em grades pequenas. O ganho aparece quando o **custo de cálculo por faixa supera o custo de comunicação** (grades grandes / mais gerações).
- **Sincronização de fronteiras:** montar e enviar as ghost rows corretamente (e tratar os workers de borda, que tem ghost de um lado só) foi o ponto mais delicado.
- **Barreira e parada:** garantir que todas as threads terminem a geração antes da troca de grades, e que a condição de parada seja avaliada uma única vez por geração.

Conclusão: o paralelismo de CPU real, em Python puro, vem da **distribuição em processos**; threads servem para estruturar a divisão e para I/O. Em linguagens sem GIL (ou com multiprocessing), a versão por faixas escalaria com os núcleos.

9. Melhorias Implementadas

Modelo estendido em FakeNews_melhorias.py (justificativa na secao 10):

- **Influenciadores digitais:** contas de grande alcance pesam 2 ao espalhar e sao mais suscetiveis (limiar -1). Inspirado em maximizacao de influencia em redes (Kempe et al., 2003).
- **Resistentes a propagacao:** individuos com letramento midiatico tem limiar dobrado, funcionando como freio. Inspirado em 'stiflers' do modelo de rumores (Daley & Kendall, 1965).
- **Estatisticas adicionais:** pico de espalhadores e geracao do pico, alcance total (quem chegou a acreditar) e nº de nunca-atingidos.
- **Visualizacao e benchmark automatizado:** geracao de graficos e CSV reprodutíveis.

10. Referencias Bibliograficas

- DALEY, D. J.; KENDALL, D. G. Stochastic rumours. *IMA Journal of Applied Mathematics*, v. 1, n. 1, p. 42-55, 1965.
- KERMACK, W. O.; McKENDRICK, A. G. A contribution to the mathematical theory of epidemics. *Proc. Royal Society A*, v. 115, 1927. (modelo SIR)
- KEMPE, D.; KLEINBERG, J.; TARDOS, E. Maximizing the spread of influence through a social network. *ACM SIGKDD*, 2003.
- BARABASI, A.-L.; ALBERT, R. Emergence of scaling in random networks. *Science*, v. 286, 1999.
- WOLFRAM, S. *A New Kind of Science*. Wolfram Media, 2002. (automatos celulares)
- TANENBAUM, A. S.; VAN STEEN, M. *Distributed Systems: Principles and Paradigms*. 3. ed. 2017.
- BEAZLEY, D. Understanding the Python GIL. *PyCon*, 2010.
- Python Software Foundation. *threading* e *socket* — documentacao oficial. docs.python.org.

11. Contribuicao Individual dos Integrantes

Preencher com o nome e a contribuicao de cada integrante:

Integrante	Principais contribuicoes
_____	_____
_____	_____
_____	_____
_____	_____

Repositorio GitHub: github.com/_____/_____ (inserir link)